Project 64: Sound Classification (UrbanSound8K)

Goal: Build a CNN or CNN-RNN model that classifies 10 common environmental sounds from the **UrbanSound8K** dataset.

🎧 UrbanSound8K Dataset Overview

- URL: https://urbansounddataset.weebly.com
- 8,732 audio files labeled with:
    - `air_conditioner`, `car_horn`, `children_playing`, `dog_bark`, `drilling`, `engine_idling`, `gun_shot`, `jackhammer`, `siren`, `street_music`

Step-by-Step Implementation

Step 1: Install Required Libraries

```
pip install librosa pandas numpy matplotlib torch torchaudio scikit-learn
```

Step 2: Extract Audio Features (MFCCs)

```
import librosa
import numpy as np

def extract_mfcc(file_path, n_mfcc=40, max_pad_len=174):
    audio, sr = librosa.load(file_path, res_type='kaiser_fast')
    mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=n_mfcc)
```

```
    pad_width = max_pad_len - mfcc.shape[1]
    if pad_width > 0:
        mfcc = np.pad(mfcc, pad_width=((0, 0), (0, pad_width)), mode='constant
    else:
        mfcc = mfcc[:, :max_pad_len]
    return mfcc
```

Step 3: Load Metadata and Prepare Dataset

```
import pandas as pd
import os

metadata = pd.read_csv("UrbanSound8K/metadata/UrbanSound8K.csv")
labels = metadata['class'].unique()
label_to_index = {label: idx for idx, label in enumerate(labels)}

features = []
targets = []

for index, row in metadata.iterrows():
    file_path = f"UrbanSound8K/audio/fold{row['fold']}/{row['slice_file_name']
    mfcc = extract_mfcc(file_path)
    features.append(mfcc)
    targets.append(label_to_index[row['class']])
```

Step 4: Build and Train a CNN Model

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

X = torch.tensor(np.array(features)).unsqueeze(1).float()  # (N, 1, 40, 174)
y = torch.tensor(np.array(targets))
```

```
train_ds = TensorDataset(X, y)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

class SoundCNN(nn.Module):
    def __init__(self):
        super(SoundCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
        self.pool = nn.MaxPool2d(2)
        self.fc1 = nn.Linear(32 * 19 * 86, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

model = SoundCNN()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

Step 5: Train the Model

```
for epoch in range(10):
    model.train()
    total_loss = 0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        out = model(xb)
        loss = criterion(out, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")
```

Step 6: Make Predictions

```
def predict(file_path):
    model.eval()
    mfcc = extract_mfcc(file_path)
    mfcc_tensor = torch.tensor(mfcc).unsqueeze(0).unsqueeze(0).float()
    with torch.no_grad():
        pred = model(mfcc_tensor)
    predicted_index = torch.argmax(pred, dim=1).item()
    return labels[predicted_index]

print("Predicted class:", predict("UrbanSound8K/audio/fold1/100032-3-0-0.wav")
```

Key Concepts

- **MFCC**: Captures audio features compactly
- **CNN**: Detects frequency-time patterns
- **UrbanSound8K**: Standard dataset for environmental sound recognition

---

Project Summary

- **Input**: Audio clip (≤4 sec)
- **Output**: One of 10 environment-related classes
- **Model**: CNN trained on MFCC features
- **Use Case**: Smart cities, surveillance, accessibility, IoT sensors